Real-Time Airspace Copilot with Agentic Multi-Agent System

Technical Report

1. Introduction & Problem Statement

1.1 Problem Statement

The global aviation industry manages thousands of flights simultaneously, requiring continuous monitoring to ensure safety and operational efficiency. Traditional airspace monitoring systems face several challenges:

1. **Information Overload**: Operations centers receive vast amounts of raw flight data that is difficult to process and analyze in real-time.

2. **Limited User-Friendly Access**: Travelers lack easy access to detailed, real-time information about their specific flights beyond basic tracking apps.

3. **Anomaly Detection Complexity**: Identifying unusual flight patterns requires human expertise and constant attention, making it resource-intensive.

4. **Fragmented Systems**: Different stakeholders (operators and travelers) need different views of the same data, but current systems don't provide unified, intelligent interfaces.

1.2 System Overview

This project presents an **Agentic Multi-Agent System** that addresses these gaps by:

- **Automating Data Collection**: Using n8n workflows to continuously fetch and process live flight data from the OpenSky Network API.

- **Providing Intelligent Analysis**: Utilizing AI agents to automatically detect anomalies, summarize airspace conditions, and answer natural language queries.

- **Enabling Dual Perspectives**: Serving both operational staff (through region monitoring) and travelers (through personal flight tracking) with contextually appropriate information.

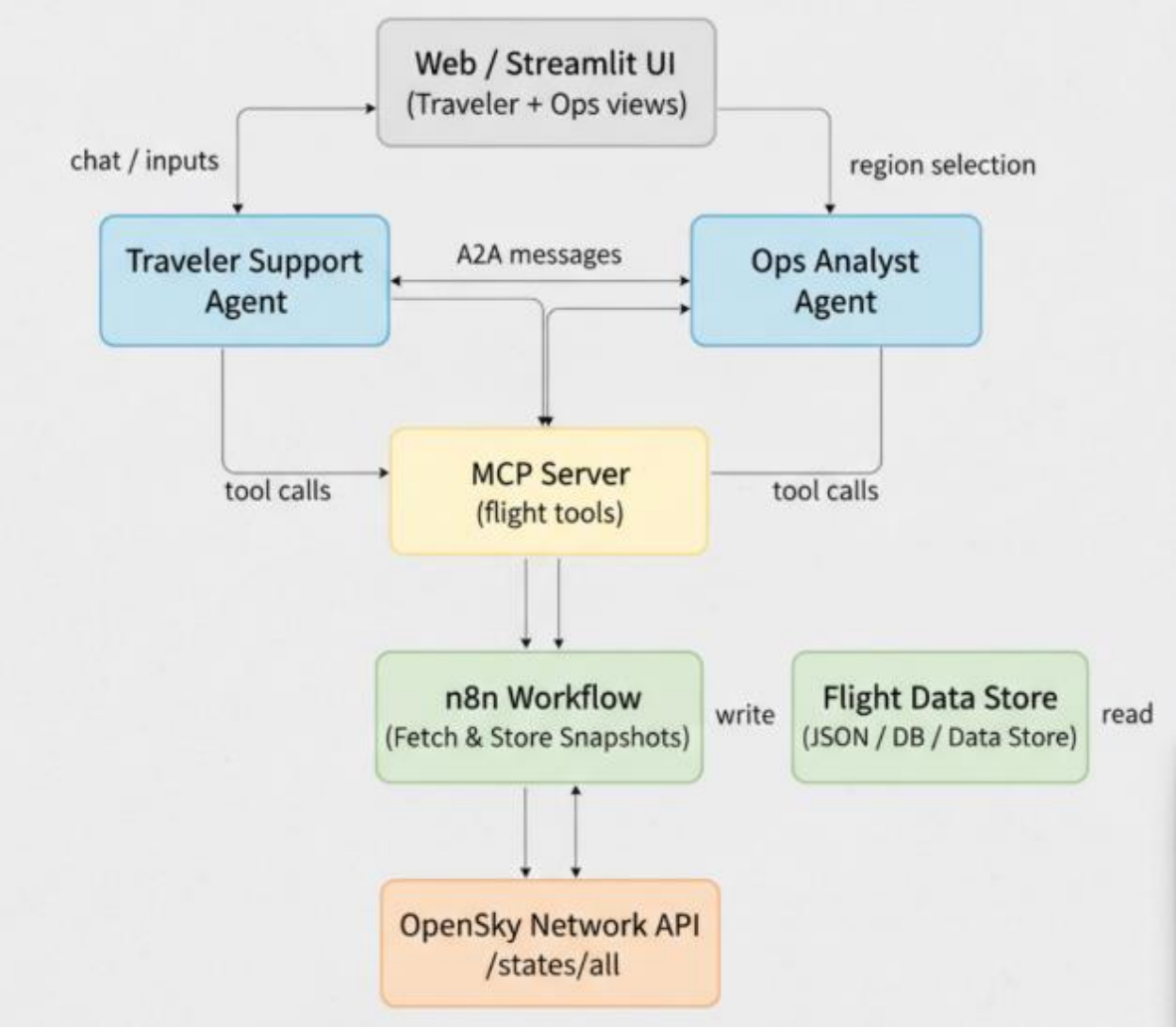
- **Ensuring Reliability**: Implementing robust error handling and fallback mechanisms to maintain service even when external APIs are unavailable.

1.3 System Architecture Overview

The system consists of four main layers:

1. **Data Layer**: n8n workflows for data fetching and storage
2. **Interface Layer**: MCP (Model Context Protocol) server exposing flight data as tools
3. **Intelligence Layer**: CrewAI-based multi-agent system for analysis and reasoning
4. **Presentation Layer**: Streamlit frontend providing user interfaces

This architecture ensures separation of concerns, scalability, and maintainability while leveraging modern AI capabilities for intelligent automation.



2. n8n Workflows and Snapshot Storage

2.1 Workflow Design

The system uses n8n, a workflow automation platform, to handle all data collection and preprocessing tasks. Two separate workflows are implemented to ensure reliability and modularity.

2.1.1 Workflow 1: Scheduled Data Fetching

Purpose: Automatically fetch and store flight data from the OpenSky Network API at regular intervals.**Workflow Structure**:

1. **Schedule Trigger**: Configured to execute every 1 minute
   - Uses n8n's Cron expression: `* * * * *`
   - Ensures fresh data availability while respecting API rate limits
2. **HTTP Request Node**: Fetches flight states from OpenSky API
   - Endpoint: `https://opensky-network.org/api/states/all`
   - Method: GET
   - Handles rate limiting (HTTP 429) gracefully
3. **Code Node (JavaScript)**: Preprocesses raw API response
   - Extracts relevant fields (callsign, ICAO24, position, altitude, velocity, etc.)
   - Filters out invalid or incomplete records
   - Structures data for downstream processing
4. **Read/Write Files Node**: Writes processed snapshot to disk
   - Location: `C:/Users/hp/Documents/data_latest.json`
   - Format: JSON with timestamp and flight array
   - Overwrites previous snapshot to maintain latest state
5. **Code Node 2 (JavaScript)**: Final formatting
   - Adds metadata (snapshot time, region identifier)

- Ensures consistent JSON structure
- Outputs formatted data for next workflow

2.1.2 Workflow 2: Webhook Endpoint

Purpose: Expose stored flight data via HTTP endpoint for system access.**Workflow Structure**:

1. **Webhook Node**: Receives HTTP GET requests
   - Path: `/webhook/latest-data`
   - Method: GET
   - No authentication required (local development)
2. **Read File Node**: Loads latest snapshot from disk
   - Reads from: `C:/Users/hp/Documents/data_latest.json`
   - Uses forward slash path format for Windows compatibility
   - Handles file read errors gracefully
3. **Code Node**: Formats response
   - Ensures valid JSON structure
   - Adds response headers
   - Prepares data for HTTP response

2.3 Error Handling and Reliability

The workflow design includes robust error handling:

- **API Failures**: If OpenSky API returns 429 (rate limit) or other errors, the system continues using the last successfully fetched snapshot
- **File Operations**: Both read and write operations include error handling to prevent workflow failures
- **Data Validation**: JavaScript code nodes validate data before storage to ensure consistency.

3. MCP Server and Tools

3.1 Model Context Protocol (MCP) Overview

The MCP Server acts as a standardized interface between the data layer (n8n) and the intelligence layer (agents). It follows the Model Context Protocol, which defines a consistent way for AI agents to access external tools and data sources.**Technology**: FastAPI (Python web framework)**Port**: 8000

3.2 MCP Tools Implementation

The server implements three core tools as required:

Tool 1: `flights.list_region_snapshot`

- **Purpose**: Retrieves the most recent flight snapshot for a specified region
- **Data Source**: n8n webhook with file fallback
- **Returns**: JSON with snapshot data including flight count and full flight array

Tool 2: `flights.get_by_callsign`

- **Purpose**: Finds the latest record for a specific flight using its callsign
- **Features**: Case-insensitive matching, handles multiple data formats
- **Returns**: Detailed flight information (position, altitude, speed, heading)

Tool 3: `alerts.list_active`

- **Purpose**: Returns all currently flagged flight anomalies
- **Implementation**: Processes flights through anomaly detection algorithm
- **Returns**: Structured list of anomalies with severity levels and descriptions

3.3 Error Handling

The MCP server implements:

- Webhook fallback to direct file reading
- HTTP error handling for connection issues
- Data validation before returning to agents

- Comprehensive logging for debugging

4. Agents, Prompts, and A2A Communication

4.1 Agent 1: Operations Analyst Agent

Role: "Airspace Operations Analyst"**Goal**: Monitor flight traffic in assigned regions, identify anomalies, and provide clear operational summaries**Configuration**:

- LLM: Groq Llama-3.1-8b-instant
- Temperature: 0.3 (focused responses)
- Max Tokens: 500
- Tools: `list_region_snapshot`, `list_active_alerts`

Task Example: Analyzes regions, summarizes flight counts, anomalies, and overall airspace status in under 200 words.

4.2 Agent 2: Traveler Support Agent

Role: "Personal Flight Watchdog Assistant"**Goal**: Help travelers track their flights and answer questions in friendly, plain language**Configuration**:

- LLM: Groq Llama-3.1-8b-instant
- Temperature: 0.5 (conversational)
- Max Tokens: 300
- Tools: `get_flight_by_callsign`, `list_active_alerts`
- **Delegation Enabled**: Can delegate to Ops Analyst

4.3 Agent-to-Agent (A2A) Communication

Implementation: Through CrewAI's delegation system when Traveler Support Agent detects keywords like "nearby", "other flights", "issues", "problems", "anomalies", or "region".**Example Scenario**:

1. Traveler asks: "Are there any other flights near mine having problems?"
2. System detects keywords → triggers A2A
3. Traveler Support gets flight info → delegates to Ops Analyst for regional context

4. Ops Analyst checks active alerts → combines insights in final response

Benefits:

- Specialization: Each agent focuses on its expertise
- Automatic routing based on question content
- Comprehensive answers combining personal and regional context

4.4 Cost Optimization

All agents use minimal LLM parameters:

- Model: `llama-3.1-8b-instant` (smaller, faster, cheaper)
- Limited tokens (300-500 per response)
- Max 3 iterations per task
- 30-second timeout

---

5. UI Design and User Journey

5.1 Frontend Technology

Framework: Streamlit**Port**: 8501

5.2 Two-Mode Interface

Traveler Mode

- Flight callsign input
- Chat interface for questions
- Quick action buttons
- Real-time flight information display

User Journey: Enter callsign → Ask questions → Receive friendly, plain-language answers

Operations Mode

- Region selector dropdown
- "Analyze Region" button
- Operational summary display showing:
  - Total flight count
  - Anomaly statistics
  - Critical issues
  - Overall status

User Journey: Select region → Click analyze → Receive comprehensive operational summary

5.3 Design Features

- Clean, uncluttered interface
- Clear labeling and instructions
- Loading indicators during processing
- Error messages for connection issues
- Responsive layout (2-column for Traveler, sidebar for Operations)

---

6. Limitations and Future Improvements

6.1 Current Limitations

1. **API Rate Limits**: OpenSky anonymous API has strict limits
2. **Rule-Based Anomaly Detection**: Simple thresholds may miss complex patterns
3. **No Temporal Analysis**: Cannot track flight history over time
4. **Local Deployment**: All components run on localhost
5. **No Authentication**: Webhook endpoints are publicly accessible

6. **Limited Context**: Agents don't remember previous conversations

6.2 Future Improvements

1. **ML-Based Anomaly Detection**: Train models on historical patterns
2. **Temporal Tracking**: Implement time-series analysis for circling detection
3. **Conversational Memory**: Add session memory to agents
4. **Cloud Deployment**: Deploy to AWS/Azure/GCP for scalability
5. **Map Visualization**: Add interactive maps showing flight positions
6. **Real-Time Updates**: WebSocket-based push notifications
7. **Database Storage**: Replace files with PostgreSQL/MongoDB
8. **Multi-Source Integration**: Add additional flight data APIs
9. **Mobile App**: Native iOS/Android applications
10. **Advanced Monitoring**: Comprehensive logging and observability

6.3 Expected Impact

These improvements would transform the system from proof-of-concept to production-ready, enabling:

- Higher accuracy through ML-based detection
- Better user experience with memory and enhanced UI
- 24/7 reliability through cloud deployment
- Scalability for thousands of concurrent users
- Predictive insights beyond current status

Conclusion

This technical report has detailed the implementation of a Real-Time Airspace Copilot system using an agentic multi-agent architecture. The system successfully integrates n8n workflows, MCP server, CrewAI agents, and Streamlit frontend to provide intelligent airspace monitoring for both travelers and operations staff.**Key Achievements**: Automated data collection via n8n Standardized tool interface

through MCP Intelligent agents with A2A communication Dual-mode interface serving different user needs Robust error handling and fallback mechanismsThe system demonstrates practical application of agentic AI to real-world problems, providing immediate utility and a foundation for future enhancements.